Laboratorul 2

Criptanaliza cifrurilor monoalfabetice
prin analiza frecvențelor

Explicarea fiecărei linii de cod din proiectul `Lab2FrequencyAnalysis`

Varianta 10 · C# / .NET 10

1. Ce face programul

Un **cifru monoalfabetic (de substituție)** înlocuiește fiecare literă a alfabetului cu o altă literă, mereu aceeași (de ex. `e` devine mereu `V`, `t` devine mereu `W`). Cheia este o permutare a celor 26 de litere, deci există 26! ≈ $4 \cdot 10^{26}$ chei posibile – prea multe pentru a fi încercate pe rând. Totuși cifrul este slab, deoarece **păstrează frecvențele literelor**: litera cea mai frecventă din textul cifrat corespunde aproape sigur unei litere foarte frecvente din engleză (`e`, `t`, `a`, `o`…).

Programul din laborator nu sparge cifrul automat, ci este un **asistent interactiv** pentru criptanalist. El încarcă criptograma variantei 10 din `Data/variant10.txt` și oferă un meniu cu care utilizatorul:

- vede tabelul de frecvențe al literelor cifrate pus alături de frecvențele standard ale limbii engleze (opțiunea 1);
- vede cele mai frecvente cuvinte de 1–4 litere, care sunt indicii pentru `a`, `I`, `of`, `to`, `the`, `and`, `that` (opțiunea 2);
- formulează ipoteze de forma `V=e W=t` și vede imediat textul parțial decriptat (opțiunea 3);
- șterge ipotezele greșite (opțiunea 4), afișează textul, cheia sau salvează rezultatul (opțiunile 5–7).

Convenția folosită peste tot în cod: **MAJUSCULELE reprezintă litere încă cifrate, minusculele reprezintă litere deja descifrate** (afișate cu verde în consolă).

2. Structura proiectului

```
Lab2FrequencyAnalysis/
├── FrequencyAnalysis.csproj     configurarea proiectului
├── Program.cs                   punctul de intrare + meniul
├── Data/variant10.txt           criptograma (varianta 10)
├── Constants/
│   └── EnglishAlphabet.cs       alfabetul + frecvențele engleze
├── Services/                    logica (fără Console)
│   ├── FrequencyAnalyzer.cs     numărare litere și cuvinte
│   └── SubstitutionKey.cs       cheia parțială, decriptare
└── UI/                          interacțiunea cu utilizatorul
    ├── ConsolePrinter.cs        afișare tabele / text / cheie
    └── InputValidator.cs        citire și validare intrare
```

Codul este împărțit pe straturi: `Constants` conține date fixe, `Services` conține logica de criptanaliză (nu scrie nimic pe ecran), iar `UI` conține tot ce ține de consolă. `Program.cs` leagă totul: citește opțiunea din meniu și apelează funcția potrivită.

3. Fluxul de execuție

1. `Program.cs` setează consola pe UTF-8 și construiește calea către `Data/variant10.txt` (copiat lângă executabil la compilare).
2. Citește criptograma; dacă lipsește sau nu are litere, afișează o eroare și se oprește.
3. Creează un obiect `SubstitutionKey` gol – cheia care va fi completată treptat.
4. Intră în bucla meniului. La fiecare opțiune apelează `ConsolePrinter` (afișare) sau `InputValidator` (citire) împreună cu `SubstitutionKey` (modificarea cheii).
5. `ConsolePrinter` folosește `FrequencyAnalyzer` pentru statistici și `SubstitutionKey.ApplyPartial` pentru textul parțial decriptat.
6. Când cheia este bună, opțiunea 7 scrie textul clar în `decrypted.txt`; opțiunea 0 închide programul.

4. Cum se folosește pentru a sparge cifrul

Procedura tipică este: (1) se afișează frecvențele și se presupune că litera cifrată cea mai frecventă este `e`, iar a doua `t`; (2) se caută în cuvintele de 3 litere un cuvânt foarte frecvent de forma `?·?·e`/`t·?·e` – acesta este `the`, care dă și litera `h`; (3) se adaugă substituțiile cu opțiunea 3 și se citește textul parțial decriptat: fragmente precum `tHe`, `Xt`, `Xn` sugerează noi litere (`i`, `n`, `a`…); (4) ipotezele care produc cuvinte imposibile se șterg cu opțiunea 4. Procesul se repetă până când toate literele sunt descifrate și textul este citibil.

În criptograma variantei 10 de exemplu, cuvântul cifrat `wqv` apare foarte des – candidatul natural este `the`, de unde `W=t Q=h V=e`.

5. Explicarea codului linie cu linie

Pentru fiecare fișier este dat codul complet, cu numerele de linie din fișierul original, și explicația fiecărei linii. Liniile goale sunt omise.

5.1. FrequencyAnalysis.csproj

Fișierul de proiect – configurează compilarea.

Nr.	Cod	Explicație
1	`<Project Sdk="Microsoft.NET.Sdk">`	Elementul rădăcină al fișierului de proiect MSBuild. `Sdk="Microsoft.NET.Sdk"` spune că este un proiect .NET în stil SDK (aplicație consolă / bibliotecă), deci majoritatea setărilor de build sunt implicite.
3	`  <PropertyGroup>`	Deschide un grup de proprietăți de build.
4	`    <OutputType>Exe</OutputType>`	`OutputType=Exe` – proiectul se compilează ca aplicație executabilă (consolă), nu ca bibliotecă DLL.
5	`    <TargetFramework>net10.0</TargetFramework>`	`TargetFramework=net10.0` – programul rulează pe .NET 10. De aici vin funcții moderne precum `char.IsAsciiLetter`.
6	`    <ImplicitUsings>enable</ImplicitUsings>`	`ImplicitUsings=enable` – compilatorul adaugă automat `using System;`, `System.IO`, `System.Linq`, `System.Collections.Generic` etc. De aceea în `Program.cs` putem folosi `File`, `Path`, `Any(...)` fără directive `using` explicite.
7	`    <Nullable>enable</Nullable>`	`Nullable=enable` – activează analiza tipurilor referință nule. Compilatorul avertizează dacă o variabilă care poate fi `null` este folosită nesigur; de aici notația `string?`.
8	`  </PropertyGroup>`	Închide grupul de proprietăți.
10	`  <ItemGroup>`	Deschide un grup de elemente (fișiere care fac parte din proiect).
11	`    <None Update="Data\variant10.txt" CopyToOutputDirectory="PreserveNewest" />`	Fișierul `Data\variant10.txt` (criptograma) este copiat la build în folderul de ieșire (`bin/Debug/net10.0/Data/`). `PreserveNewest` = se copiază doar dacă este mai nou decât copia existentă. Este necesar deoarece programul caută fișierul lângă executabil (`AppContext.BaseDirectory`).
12	`  </ItemGroup>`	Închide grupul de elemente.
14	`</Project>`	Închide elementul `Project`.

5.2. Program.cs

Punctul de intrare – încarcă criptograma și rulează meniul interactiv.

Nr.	Cod	Explicație
1	`using System.Text;`	Importă `System.Text` pentru clasa `Encoding`.
2	`using FrequencyAnalysis.Services;`	Importă serviciile (`SubstitutionKey`, `FrequencyAnalyzer`).
3	`using FrequencyAnalysis.UI;`	Importă clasele de interfață (`ConsolePrinter`, `InputValidator`).

Nr.	Cod	Explicație
5	`Console.OutputEncoding = Encoding.UTF8;`	Setează codificarea ieșirii consolei la UTF-8, ca diacriticele românești (ă, ș, ț) și săgeata → să fie afișate corect. Programul folosește „top-level statements”: nu există o metodă `Main` explicită, compilatorul o generează automat din codul acestui fișier.
6	`Console.InputEncoding = Encoding.UTF8;`	Setează și codificarea intrării la UTF-8 (pentru a putea tasta →).
8	`string variantPath = Path.Combine(AppContext.BaseDirectory, "Data", "variant10.txt");`	Construiește calea către criptogramă: folderul executabilului (`AppContext.BaseDirectory`, de ex. `bin/Debug/net10.0/`) + `Data` + `variant10.txt`. `Path.Combine` folosește separatorul corect pentru sistemul de operare.
9	`string ciphertext = File.Exists(variantPath) ? File.ReadAllText(variantPath).Trim() : string.Empty;`	Dacă fișierul există, îl citește integral și elimină spațiile/liniile goale de la capete (`Trim`); altfel `ciphertext` este un șir gol. Expresie cu operatorul ternar `? :`.
10	`var key = new SubstitutionKey();`	Creează cheia de substituție – inițial goală (nicio literă descifrată).
12	`if (!ciphertext.Any(char.IsAsciiLetter))`	Verifică dacă textul conține măcar o literă ASCII (`Any` + `char.IsAsciiLetter`). Dacă nu (fișier lipsă/gol), programul nu are ce analiza.
13	`{`	Început de bloc (acoladă deschisă).
14	`    Console.WriteLine($"Eroare: Fișierul {variantPath} lipsește sau nu conține nicio literă A–Z.");`	Afișează eroarea cu calea fișierului.
15	`    Console.WriteLine("Scrieți criptograma în Data/variant10.txt și porniți din nou programul.");`	Explică utilizatorului ce trebuie să facă.
16	`    return;`	`return` în top-level statements = închiderea programului.
17	`}`	Sfârșit de bloc (acoladă închisă).
19	`Console.WriteLine($"Criptograma a fost încărcată din Data/variant10.txt ({FrequencyAnalyzer.CountLetters(ciphertext)} litere).\n");`	Mesaj de confirmare: criptograma a fost încărcată și câte litere conține (`FrequencyAnalyzer.CountLetters`). `$"..."` este interpolare de șiruri, `\n` adaugă o linie goală.
21	`while (true)`	Bucla principală a meniului – rulează până când utilizatorul alege `0`.
22	`{`	Început de bloc (acoladă deschisă).
23	`    Console.WriteLine("1. Frecvența literelor (comparație cu limba engleză)");`	Opțiunea 1: tabelul de frecvențe comparat cu engleza.
24	`    Console.WriteLine("2. Cele mai frecvente cuvinte scurte");`	Opțiunea 2: cele mai frecvente cuvinte scurte.
25	`    Console.WriteLine("3. Adăugare substituții (ex: V=e W=t)");`	Opțiunea 3: adăugarea unor substituții (ipoteze).
26	`    Console.WriteLine("4. Ștergere substituție");`	Opțiunea 4: ștergerea unei substituții greșite.
27	`    Console.WriteLine("5. Afișare text parțial decriptat");`	Opțiunea 5: afișarea textului parțial decriptat.
28	`    Console.WriteLine("6. Afișare cheie curentă");`	Opțiunea 6: afișarea cheii curente.
29	`    Console.WriteLine("7. Salvare text decriptat în fișier");`	Opțiunea 7: salvarea textului decriptat într-un fișier.
30	`    Console.WriteLine("0. Ieșire");`	Opțiunea 0: ieșire.
31	`    Console.Write("\nAlegeți o opțiune: ");`	Cererea de alegere (`Write` – cursorul rămâne pe același rând).
33	`    string? option = Console.ReadLine();`	Citește opțiunea ca text (`string?` – poate fi `null`).

Nr.	Cod	Explicație
34	`    Console.WriteLine();`	Linie goală pentru lizibilitate.
36	`    if (option == "0")`	Dacă utilizatorul a ales `0`…
37	`    {`	Început de bloc (acoladă deschisă).
38	`        Console.WriteLine("La revedere!");`	…afișează mesajul de rămas-bun…
39	`        break;`	…și `break` iese din bucla `while`, deci programul se termină.
40	`    }`	Sfârșit de bloc (acoladă închisă).
42	`    switch (option)`	`switch` pe textul opțiunii alege acțiunea.
43	`    {`	Început de bloc (acoladă deschisă).
44	`        case "1":`	Cazul `1`:
45	`            ConsolePrinter.PrintLetterFrequencies(ciphertext, key);`	Afișează tabelul de frecvențe al criptogramei (cu substituțiile deja cunoscute).
46	`            break;`	`break` – iese din `switch`, revenind la meniu.
47	`        case "2":`	Cazul `2`:
48	`            ConsolePrinter.PrintFrequentWords(ciphertext, key);`	Afișează cuvintele scurte frecvente și forma lor parțial decriptată.
49	`            break;`	Ieșire din `switch`.
50	`        case "3":`	Cazul `3`:
51	`        {`	Acolada deschide un bloc propriu, astfel încât variabilele locale (`error`) să aibă domeniul limitat la acest caz.
52	`            foreach (var (cipher, plain) in InputValidator.ReadSubstitutions())`	Citește lista de substituții validate și o parcurge, deconstruind fiecare tuplu în `cipher` și `plain`.
53	`            {`	Început de bloc (acoladă deschisă).
54	`                string? error = key.Set(cipher, plain);`	Încearcă adăugarea substituției în cheie; `error` este `null` la succes sau mesajul de eroare (literă clară deja folosită).
55	`                Console.WriteLine(error == null`	Afișează rezultatul; operatorul ternar alege mesajul…
56	`                    ? $"  Adăugat: {cipher} → {plain}"`	…„Adăugat: V → e” la succes…
57	`                    : $"  Eroare: {error}");`	…sau „Eroare: …” la eșec.
58	`            }`	Sfârșitul `foreach`.
59	`            Console.WriteLine();`	Linie goală.
60	`            ConsolePrinter.PrintPartialText(ciphertext, key);`	După fiecare adăugare se afișează imediat textul parțial decriptat, pentru a verifica dacă ipotezele formează cuvinte englezești.
61	`            if (key.IsComplete)`	Dacă toate cele 26 de litere sunt descifrate…
62	`            {`	Început de bloc (acoladă deschisă).
63	`                Console.WriteLine("Toate cele 26 de litere au fost descifrate!\n");`	…se afișează mesajul de felicitare.
64	`            }`	Sfârșit de bloc (acoladă închisă).
65	`            break;`	Ieșire din `switch`.
66	`        }`	Sfârșitul blocului cazului 3.
67	`        case "4":`	Cazul `4`:

Nr.	Cod	Explicație
68	`        {`	Început de bloc.
69	`            char letter = InputValidator.ReadLetter("Introduceți litera criptată a cărei substituție se șterge: ");`	Citește litera cifrată (validată) a cărei substituție trebuie ștearsă.
70	`            Console.WriteLine(key.Remove(letter)`	`key.Remove(letter)` șterge substituția și returnează `true`/`false`; ternarul alege mesajul…
71	`                ? $"Substituția pentru '{letter}' a fost ștearsă.\n"`	…de succes…
72	`                : $"Eroare: Litera '{letter}' nu are nicio substituție.\n");`	…sau de eroare, dacă litera nu avea substituție.
73	`            break;`	Ieșire din `switch`.
74	`        }`	Sfârșitul blocului cazului 4.
75	`        case "5":`	Cazul `5`:
76	`            ConsolePrinter.PrintPartialText(ciphertext, key);`	Afișează criptograma parțial decriptată.
77	`            break;`	Ieșire din `switch`.
78	`        case "6":`	Cazul `6`:
79	`            ConsolePrinter.PrintKey(key);`	Afișează cheia curentă (criptare și decriptare).
80	`            break;`	Ieșire din `switch`.
81	`        case "7":`	Cazul `7`:
82	`        {`	Început de bloc.
83	`            string outputPath = Path.Combine(Environment.CurrentDirectory, "decrypted.txt");`	Calea fișierului de ieșire: `decrypted.txt` în folderul curent de lucru (de unde a fost pornit programul, de ex. folderul proiectului la `dotnet run`), nu în `bin`.
84	`            File.WriteAllText(outputPath, key.Decrypt(ciphertext) + Environment.NewLine);`	Decriptează textul cu cheia curentă și îl scrie în fișier (suprascriind un fișier existent), cu o linie nouă la final.
85	`            Console.WriteLine($"Textul decriptat a fost salvat în: {outputPath}");`	Confirmă calea unde s-a salvat.
86	`            if (!key.IsComplete)`	Dacă cheia nu este completă…
87	`            {`	Început de bloc (acoladă deschisă).
88	`                Console.WriteLine($"Atenție: cheia are doar {key.Count} din 26 de litere; literele necunoscute au rămas criptate.");`	…avertizează că literele necunoscute au rămas criptate în fișier.
89	`            }`	Sfârșit de bloc (acoladă închisă).
90	`            Console.WriteLine();`	Linie goală.
91	`            break;`	Ieșire din `switch`.
92	`        }`	Sfârșitul blocului cazului 7.
93	`        default:`	`default` – orice altă valoare introdusă:
94	`            Console.WriteLine("Opțiune invalidă! Alegeți o cifră de la 0 la 7.\n");`	Mesaj de opțiune invalidă.
95	`            break;`	Ieșire din `switch`; bucla reia meniul.
96	`    }`	Sfârșitul `switch`.
97	`}`	Sfârșitul buclei `while` și al programului.

5.3. Constants/EnglishAlphabet.cs

Alfabetul englez și frecvențele standard ale literelor.

Nr.	Cod	Explicație
1	`namespace FrequencyAnalysis.Constants;`	Declarație de namespace „file-scoped” (C# 10): tot ce urmează în fișier aparține spațiului de nume `FrequencyAnalysis.Constants`, fără acolade suplimentare.
3	`public static class EnglishAlphabet`	Clasă `public static` – nu poate fi instanțiată, conține doar membri statici. Servește drept „depozit” de constante despre alfabetul englez.
4	`{`	Început de bloc (acoladă deschisă).
5	`public const string Letters = "ABCDEFGHIJKLMNOPQRSTUVWXYZ";`	Constanta `Letters` – cele 26 de litere ale alfabetului englez, în ordine. Poziția unei litere în acest șir (0–25) este indexul ei.
6	`public const int Length = 26;`	Constanta `Length = 26` – numărul de litere; folosit pentru dimensiunea tablourilor și pentru a verifica dacă cheia este completă.
7	`public static readonly double[] Frequencies =`	Tabloul `Frequencies` – frecvențele standard (în procente) ale literelor în textele englezești. Este `static readonly`, nu `const`, deoarece în C# tablourile nu pot fi constante; `readonly` împiedică înlocuirea referinței.
8	`{`	Începutul inițializatorului de tablou.
9	`8.17, 1.49, 2.78, 4.25, 12.70, 2.23, 2.01, 6.09, 6.97, 0.15, 0.77, 4.03, 2.41,`	Frecvențele pentru literele A … M, în ordine: A=8.17, B=1.49, C=2.78, D=4.25, **E=12.70 (cea mai frecventă literă în engleză)**, F=2.23, G=2.01, H=6.09, I=6.97, J=0.15, K=0.77, L=4.03, M=2.41.
10	`6.75, 7.51, 1.93, 0.09, 5.99, 6.33, 9.06, 2.76, 0.98, 2.36, 0.15, 1.97, 0.07`	Frecvențele pentru N … Z: N=6.75, O=7.51, P=1.93, Q=0.09, R=5.99, S=6.33, **T=9.06 (a doua ca frecvență)**, U=2.76, V=0.98, W=2.36, X=0.15, Y=1.97, Z=0.07. Suma tuturor valorilor ≈ 100%.
11	`};`	Sfârșitul inițializatorului de tablou.
13	`public static int IndexOf(char c) => Letters.IndexOf(char.ToUpperInvariant(c));`	`IndexOf(c)` – metodă cu corp-expresie (`=>`). Transformă caracterul în majusculă cu `ToUpperInvariant` (independent de setările regionale) și returnează poziția lui în `Letters` (0–25), sau `-1` dacă nu este literă engleză. Astfel 'e' și 'E' au același index 4.
15	`public static bool IsLetter(char c) => IndexOf(c) >= 0;`	`IsLetter(c)` – `true` dacă caracterul este o literă A–Z / a–z (adică indexul este ≥ 0). Cifrele, spațiile, punctuația și literele cu diacritice returnează `false`.
17	`public static string LettersByFrequency =>`	Proprietatea calculată `LettersByFrequency` – returnează alfabetul englez sortat descrescător după frecvența standard.
18	`new string(Letters.OrderByDescending(c => Frequencies[IndexOf(c)]).ToArray());`	`OrderByDescending` (LINQ) sortează literele după `Frequencies[IndexOf(c)]`; `ToArray()` produce un `char[]`, iar `new string(...)` îl transformă în șir. Rezultatul este `ETAOINSHRDLCUMWFGYPBVKJXQZ` (J și X au aceeași frecvență; sortarea LINQ este stabilă, deci rămân în ordine alfabetică).
19	`}`	Sfârșit de bloc (acoladă închisă).

5.4. Services/FrequencyAnalyzer.cs

Calculul frecvențelor literelor și al cuvintelor frecvente.

Nr.	Cod	Explicație
1	`using System.Text.RegularExpressions;`	Importă `System.Text.RegularExpressions` pentru clasa `Regex` (expresii regulate), folosită la extragerea cuvintelor.
2	`using FrequencyAnalysis.Constants;`	Importă namespace-ul cu clasa `EnglishAlphabet`.
4	`namespace FrequencyAnalysis.Services;`	Toate tipurile din fișier aparțin namespace-ului `FrequencyAnalysis.Services` (logica de criptanaliză).
6	`public record LetterFrequency(char Letter, int Count, double Percent);`	`record LetterFrequency(char Letter, int Count, double Percent)` – un „record” pozițional: o clasă imutabilă generată automat de compilator, cu proprietățile `Letter` (litera), `Count` (numărul de apariții) și `Percent` (procentul din totalul literelor), plus constructor, `Equals`, `ToString` și deconstrucție.
8	`public static class FrequencyAnalyzer`	Clasă statică ce conține funcțiile de analiză statistică a textului cifrat.
9	`{`	Început de bloc (acoladă deschisă).
10	`    public static int CountLetters(string text) => text.Count(EnglishAlphabet.IsLetter);`	`CountLetters` – numără câte caractere din text sunt litere. Un `string` este o secvență de `char`, deci se poate aplica LINQ `Count(predicat)`; `EnglishAlphabet.IsLetter` este transmis ca „method group” (echivalent cu `c => EnglishAlphabet.IsLetter(c)`).
12	`    public static List<LetterFrequency> LetterFrequencies(string text)`	`LetterFrequencies` – calculează, pentru fiecare din cele 26 de litere, numărul de apariții și procentul. Returnează o listă în ordine alfabetică A…Z.
13	`    {`	Început de bloc (acoladă deschisă).
14	`        int[] counts = new int[EnglishAlphabet.Length];`	Tablou `counts` de 26 de întregi, inițializat automat cu 0 – câte un contor pentru fiecare literă.
15	`        foreach (char c in text)`	Parcurge fiecare caracter al textului.
16	`        {`	Început de bloc (acoladă deschisă).
17	`            int index = EnglishAlphabet.IndexOf(c);`	Află indexul literei (0–25) sau -1 dacă nu este literă.
18	`            if (index >= 0)`	Doar literele sunt numărate; spațiile, cifrele și punctuația sunt ignorate.
19	`            {`	Început de bloc (acoladă deschisă).
20	`                counts[index]++;`	Incrementează contorul literei. Deoarece `IndexOf` lucrează cu majuscule, 'v' și 'V' se numără împreună (numărarea nu ține cont de majuscule/minuscule).
21	`            }`	Sfârșit de bloc (acoladă închisă).
22	`        }`	Sfârșit de bloc (acoladă închisă).
24	`        int total = counts.Sum();`	`total` – numărul total de litere (suma contoarelor), folosit la calculul procentelor.
25	`        var result = new List<LetterFrequency>(EnglishAlphabet.Length);`	Creează lista rezultat cu capacitatea inițială 26 (evită realocările interne).
26	`        for (int i = 0; i < EnglishAlphabet.Length; i++)`	Parcurge toate cele 26 de litere.

Nr.	Cod	Explicație
27	`        {`	Început de bloc (acoladă deschisă).
28	`            double percent = total == 0 ? 0 : 100.0 * counts[i] / total;`	Procentul literei: `100.0 * count / total`. Scrierea `100.0` forțează împărțirea cu virgulă mobilă (altfel ar fi împărțire întreagă). Dacă textul nu are litere (`total == 0`) se pune 0, evitând împărțirea la zero.
29	`            result.Add(new LetterFrequency(EnglishAlphabet.Letters[i], counts[i], percent));`	Adaugă în listă un `LetterFrequency` cu litera, numărul de apariții și procentul.
30	`        }`	Sfârșit de bloc (acoladă închisă).
32	`        return result;`	Returnează lista celor 26 de frecvențe.
33	`    }`	Sfârșit de bloc (acoladă închisă).
35	`    public static List<LetterFrequency> SortedByFrequency(string text) =>`	`SortedByFrequency` – aceleași frecvențe, dar sortate pentru analiză.
36	`        LetterFrequencies(text)`	Pornește de la lista alfabetică produsă de `LetterFrequencies`.
37	`            .OrderByDescending(f => f.Count)`	Sortează descrescător după numărul de apariții – cea mai frecventă literă cifrată primește rangul 1.
38	`            .ThenBy(f => f.Letter)`	La egalitate de apariții, literele sunt ordonate alfabetic (rezultat determinist).
39	`            .ToList();`	`ToList()` materializează rezultatul LINQ într-o listă.
41	`    public static List<(string Word, int Count)> MostFrequentWords(string text, int length, int top)`	`MostFrequentWords(text, length, top)` – returnează cele mai frecvente `top` cuvinte de lungime exact `length`, ca listă de tupluri cu nume `(Word, Count)`.
42	`    {`	Început de bloc (acoladă deschisă).
43	`        return Regex.Matches(text.ToUpperInvariant(), "[A-Z]+")`	Textul este trecut în majuscule, apoi `Regex.Matches` găsește toate secvențele maximale de litere (`[A-Z]+`). Fiecare secvență este un „cuvânt”; spațiile, punctuația și cifrele sunt separatori.
44	`            .Select(m => m.Value)`	Din fiecare potrivire (`Match`) se păstrează doar textul (`m.Value`).
45	`            .Where(w => w.Length == length)`	Se păstrează doar cuvintele cu lungimea cerută.
46	`            .GroupBy(w => w)`	Grupează cuvintele identice (de ex. toate aparițiile lui `WQV`).
47	`            .Select(g => (Word: g.Key, Count: g.Count()))`	Fiecare grup devine un tuplu: cuvântul și numărul de apariții.
48	`            .OrderByDescending(x => x.Count)`	Sortare descrescătoare după frecvență.
49	`            .ThenBy(x => x.Word)`	La egalitate – ordine alfabetică.
50	`            .Take(top)`	Se păstrează doar primele `top` cuvinte.
51	`            .ToList();`	Materializează rezultatul într-o listă și îl returnează.
52	`    }`	Sfârșit de bloc (acoladă închisă).
53	`}`	Sfârșit de bloc (acoladă închisă).

5.5. Services/SubstitutionKey.cs

Cheia de substituție parțială: adăugare, ștergere, decriptare.

Nr.	Cod	Explicație
1	`using System.Text;`	Importă `System.Text` pentru `StringBuilder` (construirea eficientă a șirurilor).
2	`using FrequencyAnalysis.Constants;`	Importă `EnglishAlphabet`.
4	`namespace FrequencyAnalysis.Services;`	Namespace-ul serviciilor.
6	`public class SubstitutionKey`	Clasa `SubstitutionKey` reprezintă **cheia parțială** găsită până acum de criptanalist: ce literă clară corespunde fiecărei litere cifrate. Spre deosebire de celelalte clase, aceasta are stare (se creează un obiect cu `new`).
7	`{`	Început de bloc (acoladă deschisă).
8	`    private readonly Dictionary<char, char> _cipherToPlain = new();`	Dicționarul privat `_cipherToPlain`: cheia = litera cifrată (MAJUSCULĂ), valoarea = litera clară (minusculă). `readonly` înseamnă că referința la dicționar nu se schimbă, dar conținutul poate fi modificat. `new()` este „target-typed new” – tipul se deduce din declarație.
10	`    public int Count => _cipherToPlain.Count;`	`Count` – câte litere cifrate au deja o substituție.
12	`    public bool IsComplete => Count == EnglishAlphabet.Length;`	`IsComplete` – `true` când toate cele 26 de litere au fost descifrate.
14	`    public IReadOnlyDictionary<char, char> Mappings => _cipherToPlain;`	`Mappings` – expune substituțiile doar pentru citire (`IReadOnlyDictionary`), astfel încât codul din afară să nu poată modifica dicționarul direct. (În versiunea actuală a programului nu este folosită.)
16	`    public string? Set(char cipherLetter, char plainLetter)`	`Set(cipherLetter, plainLetter)` – adaugă sau înlocuiește substituția `cipherLetter → plainLetter`. Returnează `null` la succes sau un mesaj de eroare (`string?`).
17	`    {`	Început de bloc (acoladă deschisă).
18	`        cipherLetter = char.ToUpperInvariant(cipherLetter);`	Normalizează litera cifrată la majusculă. Convenția din laborator: MAJUSCULE = text cifrat.
19	`        plainLetter = char.ToLowerInvariant(plainLetter);`	Normalizează litera clară la minusculă. Convenția: minuscule = text clar descifrat.
21	`        foreach (var (cipher, plain) in _cipherToPlain)`	Parcurge toate substituțiile existente; fiecare `KeyValuePair` este deconstruit în `(cipher, plain)`.
22	`        {`	Început de bloc (acoladă deschisă).
23	`            if (plain == plainLetter && cipher != cipherLetter)`	Verifică **bijectivitatea** cheii: un cifru monoalfabetic este o permutare, deci două litere cifrate diferite nu pot avea aceeași literă clară. Dacă `plainLetter` este deja imaginea altei litere cifrate, operația este refuzată. (Dacă este aceeași literă cifrată, e permis – practic nu se schimbă nimic.)
24	`            {`	Început de bloc (acoladă deschisă).
25	`                return $"litera clară '{plainLetter}' este deja imaginea lui '{cipher}'. Ștergeți mai întâi substituția {cipher} → {plain}.";`	Returnează mesajul de eroare, care îi spune utilizatorului ce substituție trebuie ștearsă mai întâi.
26	`            }`	Sfârșit de bloc (acoladă închisă).
27	`        }`	Sfârșit de bloc (acoladă închisă).
29	`        _cipherToPlain[cipherLetter] = plainLetter;`	Indexatorul dicționarului adaugă perechea sau o suprascrie dacă litera cifrată avea deja o substituție (de ex. `V=a` după `V=e` înlocuiește ipoteza veche).

Nr.	Cod	Explicație
30	`        return null;`	`null` = succes, nicio eroare.
31	`    }`	Sfârșit de bloc (acoladă închisă).
33	`    public bool Remove(char cipherLetter) => _cipherToPlain.Remove(char.ToUpperInvariant(cipherLetter));`	`Remove` – șterge substituția literei cifrate date (după normalizare la majusculă). `Dictionary.Remove` returnează `true` dacă litera exista, `false` altfel.
35	`    public void Clear() => _cipherToPlain.Clear();`	`Clear` – șterge toate substituțiile (resetarea cheii). Nu este apelată în programul actual.
37	`    public char? PlainOf(char cipherLetter) =>`	`PlainOf(cipherLetter)` – returnează litera clară corespunzătoare sau `null` dacă încă nu se cunoaște. Tipul `char?` (nullable) permite valoarea „lipsă”.
38	`        _cipherToPlain.TryGetValue(char.ToUpperInvariant(cipherLetter), out char plain) ? plain : null;`	`TryGetValue` caută în dicționar fără să arunce excepție; dacă găsește, `plain` primește valoarea, iar operatorul ternar returnează `plain`, altfel `null`.
40	`    public string ApplyPartial(string ciphertext)`	`ApplyPartial` – produce textul **parțial decriptat** folosit la afișare: literele cunoscute devin minuscule (clare), cele necunoscute rămân MAJUSCULE (cifrate). Așa se vede imediat ce mai este de spart.
41	`    {`	Început de bloc (acoladă deschisă).
42	`        var result = new StringBuilder(ciphertext.Length);`	`StringBuilder` cu capacitatea egală cu lungimea textului – mult mai eficient decât concatenarea repetată de șiruri.
43	`        foreach (char c in ciphertext)`	Parcurge fiecare caracter al criptogramei.
44	`        {`	Început de bloc (acoladă deschisă).
45	`            if (!EnglishAlphabet.IsLetter(c))`	Dacă nu este literă (spațiu, cifră, punctuație, linie nouă)…
46	`            {`	Început de bloc (acoladă deschisă).
47	`                result.Append(c);`	…caracterul se copiază neschimbat (în laborator doar literele sunt criptate)…
48	`                continue;`	…și se trece la următorul caracter.
49	`            }`	Sfârșit de bloc (acoladă închisă).
51	`            char? plain = PlainOf(c);`	Caută litera clară pentru litera curentă.
52	`            result.Append(plain ?? char.ToUpperInvariant(c));`	Operatorul `??`: dacă există substituție se adaugă litera clară (minusculă), altfel litera cifrată trecută în MAJUSCULĂ.
53	`        }`	Sfârșit de bloc (acoladă închisă).
55	`        return result.ToString();`	Returnează textul construit.
56	`    }`	Sfârșit de bloc (acoladă închisă).
58	`    public string Decrypt(string ciphertext)`	`Decrypt` – decriptarea „finală”, folosită la salvarea în fișier. Spre deosebire de `ApplyPartial`, păstrează majusculele/minusculele originale ale textului.
59	`    {`	Început de bloc (acoladă deschisă).
60	`        var result = new StringBuilder(ciphertext.Length);`	`StringBuilder` pentru rezultat.
61	`        foreach (char c in ciphertext)`	Parcurge fiecare caracter.
62	`        {`	Început de bloc (acoladă deschisă).
63	`            char? plain = EnglishAlphabet.IsLetter(c) ? PlainOf(c) : null;`	Pentru litere se caută substituția; pentru orice altceva `plain` este `null`.

Nr.	Cod	Explicație
64	`            if (plain is null)`	Dacă nu există substituție (sau nu e literă)…
65	`            {`	Început de bloc (acoladă deschisă).
66	`                result.Append(c);`	…se păstrează caracterul original.
67	`            }`	Sfârșit de bloc (acoladă închisă).
68	`            else`	Altfel (există literă clară):
69	`            {`	Început de bloc (acoladă deschisă).
70	`                result.Append(char.IsUpper(c) ? char.ToUpperInvariant(plain.Value) : plain.Value);`	Dacă litera cifrată originală era majusculă (de ex. începutul unei propoziții), litera clară se scrie tot cu majusculă; altfel cu minusculă. `plain.Value` extrage valoarea din `char?`.
71	`            }`	Sfârșit de bloc (acoladă închisă).
72	`        }`	Sfârșit de bloc (acoladă închisă).
74	`        return result.ToString();`	Returnează textul decriptat.
75	`    }`	Sfârșit de bloc (acoladă închisă).
77	`    public string CipherAlphabet()`	`CipherAlphabet` – construiește **cheia de criptare** în forma clasică: pentru fiecare literă clară a…z, ce literă cifrată îi corespunde.
78	`    {`	Început de bloc (acoladă deschisă).
79	`        var plainToCipher = _cipherToPlain.ToDictionary(kv => kv.Value, kv => kv.Key);`	Inversează dicționarul (literă clară → literă cifrată) cu `ToDictionary`. Este sigur, deoarece `Set` garantează că nu există două litere cifrate cu aceeași literă clară (altfel `ToDictionary` ar arunca excepție pentru chei duplicate).
80	`        var result = new StringBuilder(EnglishAlphabet.Length);`	`StringBuilder` de 26 de caractere.
81	`        foreach (char p in EnglishAlphabet.Letters.ToLowerInvariant())`	Parcurge alfabetul clar a…z.
82	`        {`	Început de bloc (acoladă deschisă).
83	`            result.Append(plainToCipher.TryGetValue(p, out char c) ? c : '?');`	Adaugă litera cifrată corespunzătoare sau `'?'` dacă încă nu este cunoscută.
84	`        }`	Sfârșit de bloc (acoladă închisă).
86	`        return result.ToString();`	Returnează alfabetul cifrat (26 de caractere).
87	`    }`	Sfârșit de bloc (acoladă închisă).
88	`}`	Sfârșit de bloc (acoladă închisă).

5.6. UI/ConsolePrinter.cs

Afișarea tabelelor, a textului parțial decriptat și a cheii.

Nr.	Cod	Explicație
1	`using FrequencyAnalysis.Constants;`	Importă `EnglishAlphabet`.
2	`using FrequencyAnalysis.Services;`	Importă serviciile (`FrequencyAnalyzer`, `SubstitutionKey`).
4	`namespace FrequencyAnalysis.UI;`	Namespace-ul interfeței cu utilizatorul (UI).
6	`public static class ConsolePrinter`	Clasă statică responsabilă doar de afișarea rezultatelor în consolă (separarea logicii de prezentare).

Nr.	Cod	Explicație
7	`{`	Început de bloc (acoladă deschisă).
8	`    public static void PrintLetterFrequencies(string ciphertext, SubstitutionKey key)`	`PrintLetterFrequencies` – afișează tabelul de frecvențe al criptogramei comparat cu frecvențele limbii engleze (opțiunea 1 din meniu). Este pasul principal al atacului prin analiza frecvențelor.
9	`    {`	Început de bloc (acoladă deschisă).
10	`        var sorted = FrequencyAnalyzer.SortedByFrequency(ciphertext);`	Frecvențele literelor din criptogramă, sortate descrescător.
11	`        string english = EnglishAlphabet.LettersByFrequency;`	Literele englezești sortate după frecvența standard (`ETAOIN…`).
13	`        Console.WriteLine($"Numărul total de litere: {FrequencyAnalyzer.CountLetters(ciphertext)}\n");`	Afișează numărul total de litere din criptogramă.
14	`        Console.WriteLine("Rang  Criptat  Apariții  %   Engleză      %   Substituție");`	Capul de tabel: rangul, litera cifrată, numărul de apariții, procentul, litera engleză cu același rang, procentul ei și substituția curentă.
15	`        Console.WriteLine("----  -------  --------  -----   -------  -----   -----------");`	Linia de separare sub capul de tabel.
16	`        for (int i = 0; i < sorted.Count; i++)`	Câte un rând pentru fiecare rang (26 de rânduri).
17	`        {`	Început de bloc (acoladă deschisă).
18	`            var f = sorted[i];`	`f` – litera cifrată de pe rangul `i`.
19	`            char eng = english[i];`	`eng` – litera engleză de pe același rang. **Ideea atacului**: litera cifrată cea mai frecventă corespunde probabil lui `e`, a doua lui `t` etc. Tabelul pune alăturat aceste ipoteze.
20	`            double engPercent = EnglishAlphabet.Frequencies[EnglishAlphabet.IndexOf(eng)];`	Frecvența standard a literei engleze `eng`.
21	`            char? plain = key.PlainOf(f.Letter);`	Substituția deja stabilită pentru litera cifrată (sau `null`).
23	`            Console.Write($"{i + 1,4}  {f.Letter,7}  {f.Count,8}  {f.Percent,5:F2}   {eng,7}  {engPercent,5:F2}   ");`	Scrie coloanele formatate: `{x,4}` = aliniat la dreapta pe 4 caractere, `{x,5:F2}` = 5 caractere cu 2 zecimale. `Write` (nu `WriteLine`) – rândul continuă cu coloana de substituție.
24	`            if (plain.HasValue)`	Dacă litera cifrată are deja o substituție…
25	`            {`	Început de bloc (acoladă deschisă).
26	`                Console.ForegroundColor = ConsoleColor.Green;`	…textul devine verde…
27	`                Console.WriteLine($"{f.Letter} → {plain}");`	…se afișează perechea, de ex. `V → e`…
28	`                Console.ResetColor();`	…și culoarea se resetează la cea implicită.
29	`            }`	Sfârșit de bloc (acoladă închisă).
30	`            else`	Altfel:
31	`            {`	Început de bloc (acoladă deschisă).
32	`                Console.WriteLine("-");`	Se afișează `-` (substituție necunoscută).
33	`            }`	Sfârșit de bloc (acoladă închisă).
34	`        }`	Sfârșit de bloc (acoladă închisă).
35	`        Console.WriteLine();`	Linie goală după tabel.
36	`    }`	Sfârșit de bloc (acoladă închisă).

Nr.	Cod	Explicație
38	`    public static void PrintFrequentWords(string ciphertext, SubstitutionKey key)`	`PrintFrequentWords` – afișează cele mai frecvente cuvinte scurte (opțiunea 2). Cuvintele scurte sunt indicii puternice: `a`, `I` (1 literă), `of`, `to`, `in` (2), `the`, `and` (3), `that`, `with` (4).
39	`    {`	Început de bloc (acoladă deschisă).
40	`        for (int length = 1; length <= 4; length++)`	Parcurge lungimile 1, 2, 3 și 4.
41	`        {`	Început de bloc (acoladă deschisă).
42	`            var words = FrequencyAnalyzer.MostFrequentWords(ciphertext, length, 8);`	Primele 8 cuvinte de lungimea curentă.
43	`            Console.WriteLine($"Cuvinte de {length} {(length == 1 ? "literă" : "litere")}:");`	Titlul secțiunii; ternarul alege forma de singular („literă”) sau plural („litere”).
44	`            foreach (var (word, count) in words)`	Parcurge cuvintele, deconstruind fiecare tuplu în `word` și `count`.
45	`            {`	Început de bloc (acoladă deschisă).
46	`                Console.Write($"  {word,-6} {count,4}  → ");`	Afișează cuvântul cifrat aliniat la stânga (`{word,-6}`) și numărul de apariții, urmate de o săgeată.
47	`                PrintPartial(key.ApplyPartial(word));`	Afișează cuvântul parțial decriptat cu cheia curentă (literele cunoscute apar verzi).
48	`                Console.WriteLine();`	Trece la rândul următor.
49	`            }`	Sfârșit de bloc (acoladă închisă).
50	`            Console.WriteLine();`	Linie goală între grupuri.
51	`        }`	Sfârșit de bloc (acoladă închisă).
52	`    }`	Sfârșit de bloc (acoladă închisă).
54	`    public static void PrintPartial(string partialText)`	`PrintPartial` – afișează un text parțial decriptat, colorând cu verde literele minuscule (cele deja descifrate).
55	`    {`	Început de bloc (acoladă deschisă).
56	`        foreach (char c in partialText)`	Parcurge caracter cu caracter.
57	`        {`	Început de bloc (acoladă deschisă).
58	`            if (char.IsLower(c))`	Minusculă = literă descifrată (conform convenției din `ApplyPartial`).
59	`            {`	Început de bloc (acoladă deschisă).
60	`                Console.ForegroundColor = ConsoleColor.Green;`	Setează culoarea verde.
61	`                Console.Write(c);`	Scrie caracterul.
62	`                Console.ResetColor();`	Revine la culoarea implicită.
63	`            }`	Sfârșit de bloc (acoladă închisă).
64	`            else`	Altfel (literă cifrată majusculă sau alt caracter):
65	`            {`	Început de bloc (acoladă deschisă).
66	`                Console.Write(c);`	Scrie caracterul fără culoare.
67	`            }`	Sfârșit de bloc (acoladă închisă).
68	`        }`	Sfârșit de bloc (acoladă închisă).
69	`    }`	Sfârșit de bloc (acoladă închisă).

Nr.	Cod	Explicație
71	`    public static void PrintPartialText(string ciphertext, SubstitutionKey key)`	`PrintPartialText` – afișează întreaga criptogramă parțial decriptată (opțiunile 3 și 5).
72	`    {`	Început de bloc (acoladă deschisă).
73	`        Console.WriteLine($"Litere descifrate: {key.Count} din {EnglishAlphabet.Length} " +`	Afișează progresul: câte litere din 26 sunt descifrate…
74	`                           "(MAJUSCULE = încă criptate, minuscule verzi = descifrate)\n");`	…și legenda culorilor. Cele două șiruri sunt concatenate cu `+`.
75	`        PrintPartial(key.ApplyPartial(ciphertext));`	Aplică cheia parțială pe criptogramă și afișează rezultatul colorat.
76	`        Console.WriteLine("\n");`	`"\n"` plus linia nouă a lui `WriteLine` = două linii noi după text.
77	`    }`	Sfârșit de bloc (acoladă închisă).
79	`    public static void PrintKey(SubstitutionKey key)`	`PrintKey` – afișează cheia curentă în ambele direcții (opțiunea 6).
80	`    {`	Început de bloc (acoladă deschisă).
81	`        string plainAlphabet = EnglishAlphabet.Letters.ToLowerInvariant();`	Alfabetul clar `abc…z`.
82	`        string cipherAlphabet = key.CipherAlphabet();`	Alfabetul cifrat corespunzător (cu `?` pentru literele necunoscute).
84	`        Console.WriteLine("Cheia de criptare (literă clară → literă criptată):");`	Titlu: cheia de criptare (clar → cifrat).
85	`        Console.WriteLine($"  Alfabet clar:    {string.Join(' ', plainAlphabet.ToCharArray())}");`	Afișează alfabetul clar; `string.Join(' ', ...)` pune un spațiu între litere, pentru aliniere.
86	`        Console.WriteLine($"  Alfabet criptat: {string.Join(' ', cipherAlphabet.ToCharArray())}");`	Afișează alfabetul cifrat sub cel clar – fiecare coloană este o pereche a cheii.
88	`        Console.WriteLine("\nCheia de decriptare (literă criptată → literă clară):");`	Titlu: cheia de decriptare (cifrat → clar).
89	`        Console.Write("  Alfabet criptat: ");`	Eticheta rândului cu alfabetul cifrat.
90	`        Console.WriteLine(string.Join(' ', EnglishAlphabet.Letters.ToCharArray()));`	Alfabetul cifrat A…Z, cu spații.
91	`        Console.Write("  Alfabet clar:    ");`	Eticheta rândului cu alfabetul clar.
92	`        Console.WriteLine(string.Join(' ', EnglishAlphabet.Letters.Select(c => key.PlainOf(c) ?? '?')));`	Pentru fiecare literă cifrată A…Z se afișează litera clară sau `'?'`. `key.PlainOf(c) ?? '?'` transformă `char?` în `char`.
94	`        Console.WriteLine($"\nLitere descifrate: {key.Count} din {EnglishAlphabet.Length}.\n");`	Afișează progresul (litere descifrate din 26).
95	`    }`	Sfârșit de bloc (acoladă închisă).
96	`}`	Sfârșit de bloc (acoladă închisă).

5.7. UI/InputValidator.cs

Citirea și validarea datelor introduse de utilizator.

Nr.	Cod	Explicație
1	`using FrequencyAnalysis.Constants;`	Importă `EnglishAlphabet` (pentru validarea literelor).
3	`namespace FrequencyAnalysis.UI;`	Namespace-ul UI.

Nr.	Cod	Explicație
5	`public static class InputValidator`	Clasă statică ce citește și validează datele introduse de utilizator.
6	`{`	Început de bloc (acoladă deschisă).
7	`public static List<(char Cipher, char Plain)> ReadSubstitutions()`	`ReadSubstitutions` – citește una sau mai multe substituții de forma `V=e W=t` și returnează o listă de tupluri `(Cipher, Plain)`.
8	`{`	Început de bloc (acoladă deschisă).
9	`while (true)`	Buclă infinită: se repetă până când utilizatorul introduce o linie validă.
10	`{`	Început de bloc (acoladă deschisă).
11	`Console.Write("Introduceți substituțiile (ex: V=e W=t): ");`	Afișează cererea, cu un exemplu de format.
12	`string rawInput = Console.ReadLine() ?? string.Empty;`	Citește linia. `ReadLine()` poate returna `null` (sfârșitul intrării), caz în care `??` pune un șir gol.
13	`string[] tokens = rawInput.Split(new[] { ' ', ',', ';' }, StringSplitOptions.RemoveEmptyEntries);`	Împarte linia în „token”-uri după spațiu, virgulă sau punct și virgulă. `RemoveEmptyEntries` ignoră separatorii multipli (de ex. două spații).
15	`if (tokens.Length == 0)`	Dacă nu s-a introdus nimic…
16	`{`	Început de bloc (acoladă deschisă).
17	`Console.WriteLine("Eroare: Nu ați introdus nicio substituție! Încercați din nou.\n");`	…se afișează eroarea…
18	`continue;`	…și se reia bucla (cerere nouă).
19	`}`	Sfârșit de bloc (acoladă închisă).
21	`var result = new List<(char, char)>();`	Lista în care se adună substituțiile valide.
22	`string? error = null;`	Mesajul de eroare; `null` = nicio eroare până acum.
23	`foreach (string token in tokens)`	Parcurge fiecare token.
24	`{`	Început de bloc (acoladă deschisă).
25	`string pair = token.Replace("=", "").Replace("->", "").Replace("→", "");`	Elimină separatorii `=`, `->` și `→`. Astfel `V=e`, `V->e`, `V→e` și chiar `Ve` devin toate `Ve` – exact două caractere.
26	`if (pair.Length != 2 || !EnglishAlphabet.IsLetter(pair[0]) || !EnglishAlphabet.IsLetter(pair[1]))`	Validare: trebuie să rămână exact 2 caractere și ambele să fie litere englezești.
27	`{`	Început de bloc (acoladă deschisă).
28	`error = $"Eroare: '{token}' nu este o substituție validă! Folosiți forma LiteraCriptată=literaClară, de exemplu V=e.\n";`	Construiește mesajul de eroare cu tokenul greșit și formatul corect.
29	`break;`	Iese din `foreach` la prima eroare.
30	`}`	Sfârșit de bloc (acoladă închisă).
32	`result.Add((char.ToUpperInvariant(pair[0]), char.ToLowerInvariant(pair[1])));`	Adaugă perechea normalizată: litera cifrată în majusculă, cea clară în minusculă.
33	`}`	Sfârșit de bloc (acoladă închisă).
35	`if (error != null)`	Dacă a apărut o eroare…
36	`{`	Început de bloc (acoladă deschisă).
37	`Console.WriteLine(error);`	…se afișează…

Nr.	Cod	Explicație
38	`continue;`	…și se cere din nou întreaga linie. Nicio substituție din linia greșită nu este aplicată (validare „totul sau nimic”).
39	`}`	Sfârșit de bloc (acoladă închisă).
41	`return result;`	Toate tokenurile sunt valide – se returnează lista.
42	`}`	Sfârșit de bloc (acoladă închisă).
43	`}`	Sfârșit de bloc (acoladă închisă).
45	`public static char ReadLetter(string prompt)`	`ReadLetter(prompt)` – citește o singură literă (folosită la ștergerea unei substituții).
46	`{`	Început de bloc (acoladă deschisă).
47	`while (true)`	Buclă până la o intrare validă.
48	`{`	Început de bloc (acoladă deschisă).
49	`Console.Write(prompt);`	Afișează mesajul primit ca parametru.
50	`string rawInput = (Console.ReadLine() ?? string.Empty).Trim();`	Citește linia (sau șir gol dacă e `null`) și elimină spațiile de la capete cu `Trim()`.
51	`if (rawInput.Length == 1 && EnglishAlphabet.IsLetter(rawInput[0]))`	Valid doar dacă este exact un caracter și acesta este literă A–Z.
52	`{`	Început de bloc (acoladă deschisă).
53	`return char.ToUpperInvariant(rawInput[0]);`	Returnează litera în majusculă (literele cifrate sunt păstrate cu majuscule).
54	`}`	Sfârșit de bloc (acoladă închisă).
56	`Console.WriteLine("Eroare: Introduceți o singură literă din alfabetul englez (A–Z).\n");`	Altfel afișează eroarea, iar bucla cere din nou.
57	`}`	Sfârșit de bloc (acoladă închisă).
58	`}`	Sfârșit de bloc (acoladă închisă).
59	`}`	Sfârșit de bloc (acoladă închisă).

5.8. Data/variant10.txt

Nu este cod, ci datele de intrare: criptograma variantei 10. Doar literele sunt criptate; spațiile, punctuația, cifrele (`14`, `1716`, `22`) și majusculele/minusculele sunt păstrate. Primele rânduri:

```
Vgjstgo, wnn, qto xwp asthl hqtzavi. Xw avjtg rxwq wqv hifuwtgtsfwxhvgovtknip nc Enqg Rtssxp,
wqv jivtwvpw Vgjsxpq ztwqvztwxhxtg avcnivGvrwng. Tcwvi qxp ovtwq, xw ovphvgovo wqindjq
qxp jitgopng wn ivthq, ngZtf 14, 1716, Vortio Rxssvp, t 22-fvti-nso zxgxpwvi tw Nixvs
Hnssvjv,Nycnio.Rxssvp vzatilvo tw nghv dung t htivvi dgxbdv xg wqv tggtsp nchifuwnsnjf tgo wqv
```

Fișierul este copiat automat în folderul de ieșire datorită liniei 11 din `FrequencyAnalysis.csproj`.

5.9. .gitignore

Fișier standard generat cu `dotnet new gitignore`. Spune lui Git ce să nu urmărească: folderele de build `bin/` și `obj/`, setările personale ale Visual Studio / Rider / VS Code (`.vs/`, `.idea/`, `*.user`), fișiere temporare și jurnale. Nu influențează programul, ci doar depozitul Git.

6. Noțiuni C# folosite în cod

Top-level statements	codul din `Program.cs` fără clasă și fără `Main`; compilatorul generează automat metoda de intrare.
Namespace file-scoped	`namespace X;` – întregul fișier aparține namespace-ului, fără acolade.
Clasă statică	nu se poate instanția; se apelează direct `FrequencyAnalyzer.CountLetters(...)`.
record	tip imutabil cu proprietăți, constructor și egalitate generate automat (`LetterFrequency`).
Tipuri nullable	`string?`, `char?` – pot conține `null`; `??` dă o valoare implicită, `.Value` extrage valoarea.
Membri cu corp-expresie	`=> expresie` în loc de `{ return expresie; }`.
LINQ	`Count`, `Any`, `Where`, `GroupBy`, `Select`, `OrderByDescending`, `ThenBy`, `Take`, `ToList` – prelucrarea colecțiilor în stil declarativ.
Tupluri și deconstrucție	`(string Word, int Count)` și `foreach (var (word, count) in ...)`.
Interpolare și formatare	`$"{x,4}"` aliniere la dreapta pe 4 caractere, `{x,-6}` la stânga, `{x:F2}` două zecimale.
Dictionary / StringBuilder	dicționarul ține cheia (literă cifrată → literă clară); `StringBuilder` construiește eficient textul decriptat.